BTEUP EXAM NOTES | HINGLISH EDITION

Mobile App Development

Unit II: Application Design Essentials: Activities to Permissions

In notes ke baare me

Part A me har topic simple Hinglish me real-life example ke saath samjhaya hai, taaki beginner bhi zero se samajh sake.

Part B me important exam questions ke poore 5-marks wale answers Hinglish me diye hain.

Last me: quick revision sheet, common mistakes aur self-test.

Exam me answer English me likhna hai to English Edition PDF use karo.

Syllabus covered (Unit II)

Android Application Design Essentials: Anatomy of an Android application, Android terminologies, Application Context, Activities, Services, Intents, Receiving and Broadcasting Intents, Android Manifest File and its common settings, Using Intent Filter, Permissions.

Unit Overview aur Contents

Simple idea: Android app ek bada program nahi hota. Ye **components ki team** hota hai: Activities (screens), Services (background kaam), Broadcast Receivers (listeners) aur Content Providers (data sharing). **Intents** wo messages hain jo inhe jodte hain, **manifest** app ka ID card hai jisme ye sab listed hote hain, **intent filters** batate hain ki kaun kya handle kar sakta hai, aur **permissions** decide karti hain ki app kya kar sakta hai. Ye subject ka **sabse important unit** hai.

Topic	Exam importance	Notes me kahan
Anatomy aur terminologies	Medium	Part A: Topic 1, 2 | Part B: Q1
Application Context	High	Part A: Topic 3 | Part B: Q2
Activities aur life cycle	Very High (almost sure)	Part A: Topic 4 | Part B: Q3, Q4
Services	Very High	Part A: Topic 5 | Part B: Q5, Q6
Intents	Very High	Part A: Topic 6 | Part B: Q7
Broadcast receivers	Very High	Part A: Topic 7 | Part B: Q8
Manifest file aur settings	Very High	Part A: Topic 8 | Part B: Q9
Intent filter	Very High	Part A: Topic 9 | Part B: Q10
Permissions	Very High	Part A: Topic 10 | Part B: Q11, Q12

Contents

Part A: Easy Explanation	Part B: Important Questions aur Answers
Topic 1: App ki anatomy Topic 2: Android terminologies Topic 3: Application Context Topic 4: Activities Topic 5: Services Topic 6: Intents Topic 7: Intents receive aur broadcast karna Topic 8: Manifest file aur common settings Topic 9: Intent filter Topic 10: Permissions	Q1 Android terminologies (Likely) Q2 Context (Likely) Q3 Activity life cycle (Very Likely) Q4 Activity code, passing data (Very Likely) Q5 Services (Very Likely) Q6 Service example (Likely) Q7 Intents (Very Likely) Q8 Broadcast receiver (Very Likely) Q9 Manifest settings (Very Likely) Q10 Intent filter (Very Likely) Q11 Permissions (Very Likely) Q12 Runtime permission code (Likely) Q13 Differences (Possible)

Is unit ko kaise padhein (time: 1 se 2 din)

1) Part A ek baar padh kar samjho. 2) **Activity life cycle diagram** bina dekhe banane tak practice karo. 3) Code snippets (activity, intent, service, receiver, permission) yaad karo. 4) Quick Revision Sheet se revise karo aur Self-Test karo.

PART A: Easy Explanation (Hinglish)

Topic 1: Anatomy of an Android Application (quick recap)

Unit I me project ke folders aur files dekhe the. Ab app ko **building blocks ke set** ki tarah dekho.

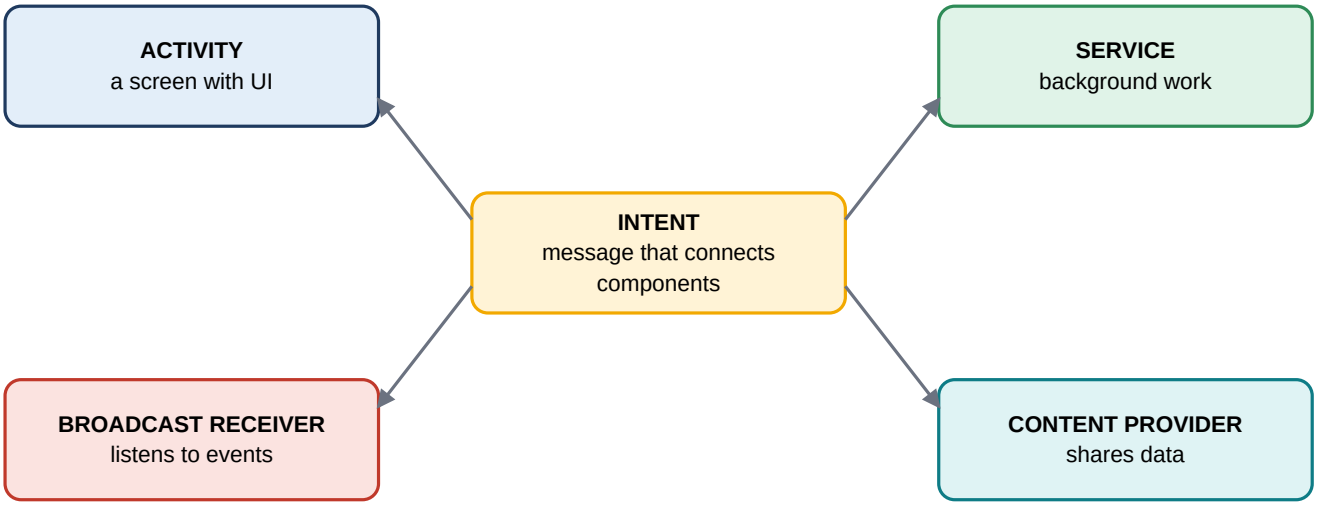


Figure 1: Four building blocks of an Android app, connected by Intents

Part	Kaam
Components	Activity, Service, Broadcast Receiver, Content Provider. Manifest me declare hote hain.
Intents	Messages jo components ko start karte hain ya unke beech data le jaate hain.
Manifest	ID card: components, permissions, features ki list.
Resources (res/)	Layouts, strings, images, colours. R class se milte hain.
Gradle files	Build settings: applicationId, minSdk, dependencies.

App icon dabane par kya hota hai

- Launcher ek **Intent** bhejta hai jisme action MAIN aur category LAUNCHER hota hai (manifest se liya hua).
- System app ke liye ek **naya Linux process** banata hai (har app apne sandbox me chalta hai).
- Pehle **Application** object banta hai, phir launcher **Activity** banti hai aur uske life-cycle methods chalte hain (onCreate, onStart, onResume).

Topic 2: Android Terminologies

Ye words baar baar aate hain. Inhe vocabulary list ki tarah seekho.

Term	Matlab
Activity	UI wali ek screen (login screen).
Service	Wo component jo bina UI ke background me kaam karta hai.
Broadcast Receiver	Wo component jo system-wide ya app messages sunta hai (battery low).
Content Provider	Wo component jo doosre apps ke saath data share karta hai (contacts).
Intent	Message jo component start karta hai ya data bhejta hai.

Term	Matlab
Intent Filter	Batata hai ki component kaunse intents handle kar sakta hai.
View / ViewGroup	View = UI element (Button, TextView). ViewGroup = views ka container (layout).
Layout	XML file jo screen par views ko arrange karti hai.
Fragment	Screen ka reusable hissa jo activity ke andar rehta hai.
Context	App environment ka handle (resources, services, system).
Manifest	AndroidManifest.xml, app ka ID card.
Resources / R class	Non-code files (layouts, strings, images); R unhe IDs deta hai.
Bundle	Key-value container jo chhota data pass ya save karne ke kaam aata hai.
Task / Back stack	User ki dekhi hui activities ka stack.
Toast	Chhota message jo screen par kuch der ke liye dikhta hai.
Notification	Status bar aur notification drawer me dikhne wala message.
Widget	Home screen par dikhne wala mini app view (UI elements ke liye bhi use hota hai).
APK / AAB	App ka installable package / publishing bundle.
AVD / ADB	Virtual device (emulator) / device se command-line bridge.
Logcat	Window jo debugging ke liye log messages dikhati hai.
Permission	Protected data ya features use karne ke liye zaroori approval.
ART / DEX	App chalane wala runtime / Android ka bytecode format.

Topic 3: Application Context

Real-life example

Context ko **access rights wale ID badge** ki tarah socho. Badge se tum kamre (resources) khol sakte ho, lift (system services) bula sakte ho aur doosre offices (activities) me ja sakte ho. Context ke bina tumhara code Android ki duniya me kuch access nahi kar sakta.

Definition

Context ek abstract class hai jo **application environment ki global information** ka interface deti hai. Isse code resources, assets, databases, system services access kar sakta hai, aur activities, services launch kar sakta hai ya broadcasts bhej sakta hai.

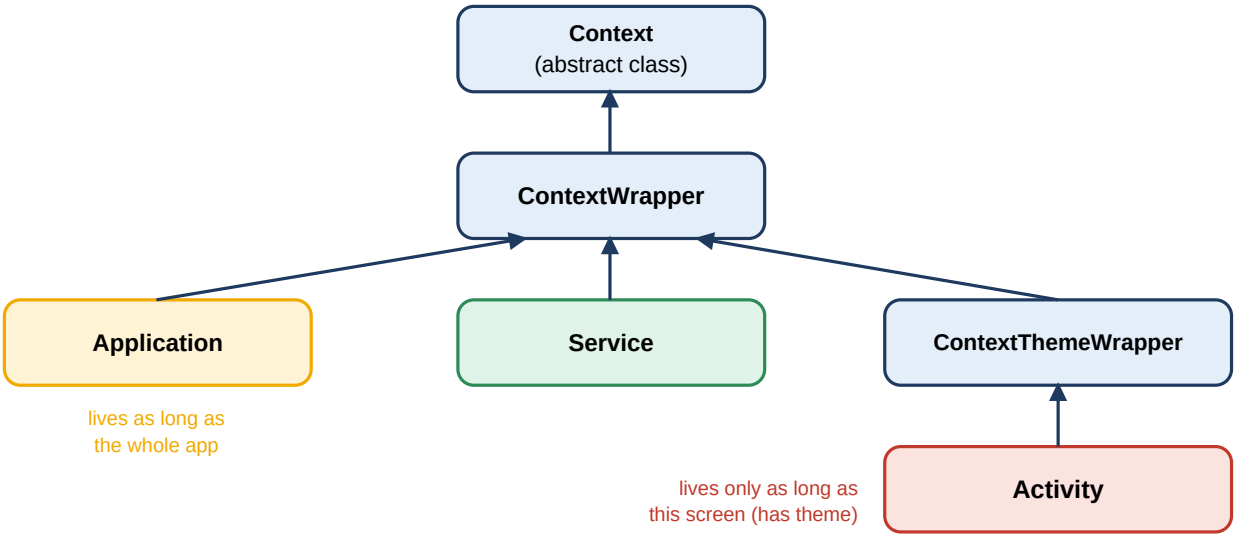


Figure 2: Context class hierarchy

Type of context	Kaise milta hai	Lifetime
Application context	getApplicationContext()	Poora app (process khatam hone tak)
Activity context	this or MainActivity.this	Jab tak wo activity hai
Service context	this (inside a Service)	Jab tak service hai
Base context	getBaseContext()	ContextWrapper ka wrapped context

Kis kaam ke liye kaunsa context?

Operation	Activity context	Application context
Dialog dikhana	Haan	Nahi (crash)
Themed layout inflate karna	Haan	Chalta hai, par default theme
Activity start karna	Haan	Sirf FLAG_ACTIVITY_NEW_TASK ke saath
Service start / broadcast bhejna	Haan	Haan
Resource load / system service use karna	Haan	Haan
Long-lived object (singleton) me rakhna	Memory leak ka risk	Safe

- **Memory leak:** agar long-lived object Activity context pakde rakhe, to activity band hone ke baad bhi memory se clear nahi ho pati. Long-lived objects ke liye **application context** use karo.
- **Application class:** `android.app.Application` app ki global state rakhti hai. Ye **kisi bhi activity, service ya receiver se pehle** banti hai. Custom class ko manifest me `android:name=".MyApp"` se register karo.

Context examples

```
// Using the activity context (this) and the application context
Toast.makeText(this, "Hello", Toast.LENGTH_SHORT).show();
Context appContext = getApplicationContext();

// A custom Application class
public class MyApp extends Application {
    @Override
    public void onCreate() {
        super.onCreate();   // runs once when the app process starts
    }
}
```

Topic 4: Activities

Real-life example

Activity **kitab ka ek page** hai. Kitab (app) me kai pages hote hain; ek time par ek hi page khula hota hai. Reader aage ja sakta hai (nayi activity) ya peeche (Back button).

Definition

Activity ek application component hai jo **ek screen** deta hai jisse user interact kar sakta hai, jaise login screen ya list. Har activity ko **manifest me declare** karna zaroori hai.

- Activity aam taur par **AppCompatActivity** ko extend karti hai, aur uska screen design XML layout se aata hai jo `setContentView()` se set hota hai.
- System activities ko callback methods se manage karta hai jinhe **life cycle** kehte hain. Ye kab chalenge, ye system decide karta hai, tum nahi.

Activity life cycle

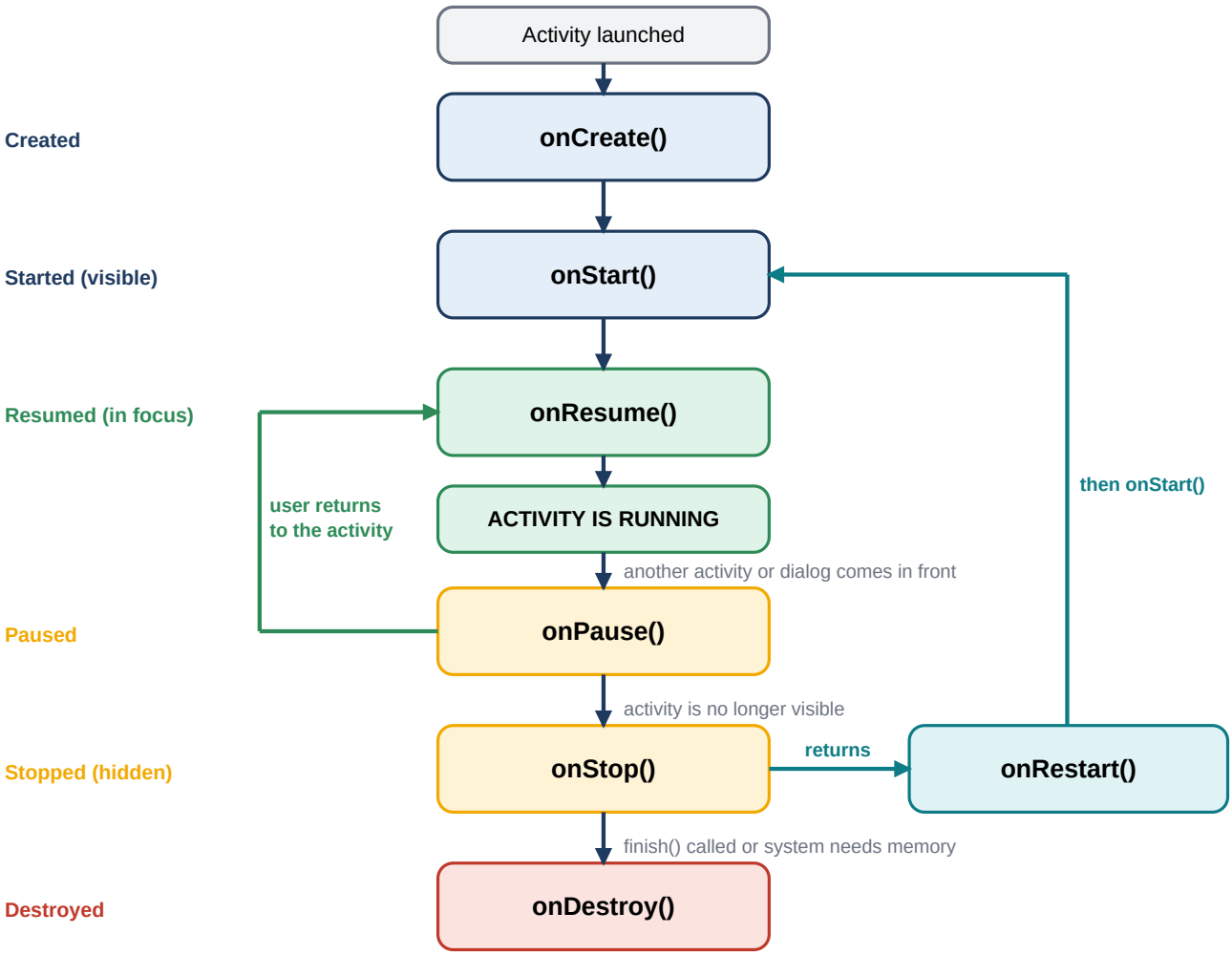


Figure 3: Activity life cycle

Method	Kab call hota hai
onCreate()	Activity banti hai (ek baar). Iske baad hamesha onStart().
onStart()	Activity **visible** hoti hai.
onResume()	Activity foreground me hai aur user interact kar sakta hai.
onPause()	Doosri activity ya dialog saamne aa raha hai; activity aanshik dikh sakti hai.
onStop()	Activity ab **dikh nahi rahi**.
onRestart()	Activity stopped state se wapas aati hai; iske baad onStart().
onDestroy()	Activity destroy ho rahi hai (finish() call hua ya system ko memory chahiye).

Yahan ye karo	Method
Layout set karo, views dhundo, data initialise karo (ek baar)	onCreate()
Animations shuru karo, visible UI chahiye wali cheezein register karo	onStart() / onResume()
Animations roko, halka data save karo, camera/sensors chhodo	onPause()
Data permanently save karo, heavy kaam roko	onStop()
Final cleanup (threads, listeners)	onDestroy()

Kab kya call hota hai? (common scenarios)

Scenario	Methods call hone ka order
App pehli baar khulta hai	onCreate, onStart, onResume
User Home button dabata hai	onPause, onStop
User app me wapas aata hai	onRestart, onStart, onResume
User Back button dabata hai (activity band)	onPause, onStop, onDestroy
Saamne dialog aata hai, phir band hota hai	onPause, then onResume
Screen rotate hoti hai (configuration change)	onPause, onStop, onDestroy, then onCreate, onStart, onResume (activity is recreated)
Activity A, Activity B start karti hai	A.onPause, B.onCreate, B.onStart, B.onResume, A.onStop

- **Teen lifetimes: entire lifetime** = onCreate se onDestroy; **visible lifetime** = onStart se onStop; **foreground lifetime** = onResume se onPause.
- **Char states:** Resumed (running), Paused (aanshik visible), Stopped (hidden par memory me), Destroyed.
- **Process death:** memory kam hone par stopped activity kill ho sakti hai, isliye **zaroori data save karo**.
- Hamesha pehle **super** method call karo (jaise `super.onCreate(savedInstanceState)`).

MainActivity.java: logging every life-cycle method (watch in Logcat)

```
public class MainActivity extends AppCompatActivity {
    private static final String TAG = "Lifecycle";

    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Log.d(TAG, "onCreate");
    }
    @Override protected void onStart()   { super.onStart();   Log.d(TAG, "onStart"); }
    @Override protected void onResume()  { super.onResume();  Log.d(TAG, "onResume"); }
    @Override protected void onPause()   { super.onPause();   Log.d(TAG, "onPause"); }
    @Override protected void onStop()    { super.onStop();    Log.d(TAG, "onStop"); }
    @Override protected void onRestart() { super.onRestart(); Log.d(TAG, "onRestart"); }
    @Override protected void onDestroy() { super.onDestroy(); Log.d(TAG, "onDestroy"); }
}
```

State save aur restore karna

Screen rotate hone par activity destroy hokar dobara banti hai, isliye variables kho jaate hain. Chhota data `onSaveInstanceState()` me **Bundle** me save karo aur `onCreate()` me wapas padho.

Saving state across rotation

```
private int count = 0;

@Override
protected void onSaveInstanceState(Bundle outState) {
    super.onSaveInstanceState(outState);
    outState.putInt("count", count);          // save
}

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    if (savedInstanceState != null) {
        count = savedInstanceState.getInt("count");   // restore
    }
}
```

Doosri activity start karna aur data pass karna

Two activities

```
// In MainActivity: start SecondActivity and send data
Button btn = findViewById(R.id.btnNext);
btn.setOnClickListener(v -> {
    Intent intent = new Intent(MainActivity.this, SecondActivity.class);
    intent.putExtra("name", "Amrit");
    startActivity(intent);
});

// In SecondActivity: receive the data
String name = getIntent().getStringExtra("name");
TextView tv = findViewById(R.id.tvName);
tv.setText("Hello " + name);

// Manifest: every activity must be declared
// <activity android:name=".SecondActivity" android:exported="false" />
```

- **finish()** current activity ko band karta hai. User pichhli activity par wapas aata hai.
- **Back stack:** har nayi activity stack par push hoti hai; Back button upar wali ko pop karta hai.

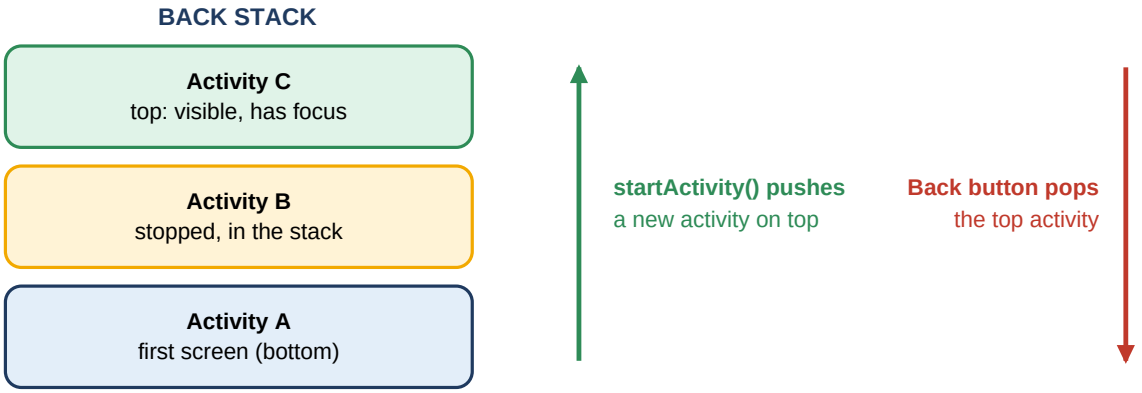


Figure 4: Back stack of activities

Launch modes (nayi instance kaise banti hai)

Mode (android:launchMode)	Behaviour
standard (default)	Har baar nayi instance banti hai, chahe pehle se ho.
singleTop	Activity agar stack ke top par pehle se hai to wahi reuse hoti hai (onNewIntent()). Nahi to nayi banti hai.
singleTask	Task me sirf ek instance; uske upar ki activities clear ho jaati hain.
singleInstance	singleTask jaisa, par activity apne alag task me akeli hoti hai.

Topic 5: Services

Real-life example

Service restaurant ke **kitchen staff** jaisi hai: ye parde ke peeche kaam karte hain jabki customers (user) sirf dining hall (screen) dekhte hain. Music screen chhodne ke baad bhi gaana bajna service ka kaam hai.

Definition

Service ek application component hai jo **background me long-running operations** kar sakta hai aur **user interface nahi deta**.

Services ke types

Type	Matlab	Example
Foreground	Aisa kaam jo user ko dikhta hai; **notification** dikhana zaroori. Kam kill hoti hai.	Music player, fitness tracker
Background	Aisa kaam jo user ko nahi dikhta. Android 8 se bahut restricted.	Storage compact karna
Bound	Doosre components **bindService()** se bind hote hain; client-server jaisa kaam; clients bound hone tak hi chalti hai.	Activity se control hone wali music service

- **Started service:** `startService()` se shuru hoti hai; tab tak chalti hai jab tak wo **stopSelf()** call na kare ya koi doosra component **stopService()** na kare.
- Service by default **main (UI) thread** par chalti hai. Heavy kaam alag thread par karo, nahi to app me **ANR** (Application Not Responding) aa sakta hai.
- Ek service started aur bound dono ho sakti hai.

Service life cycle

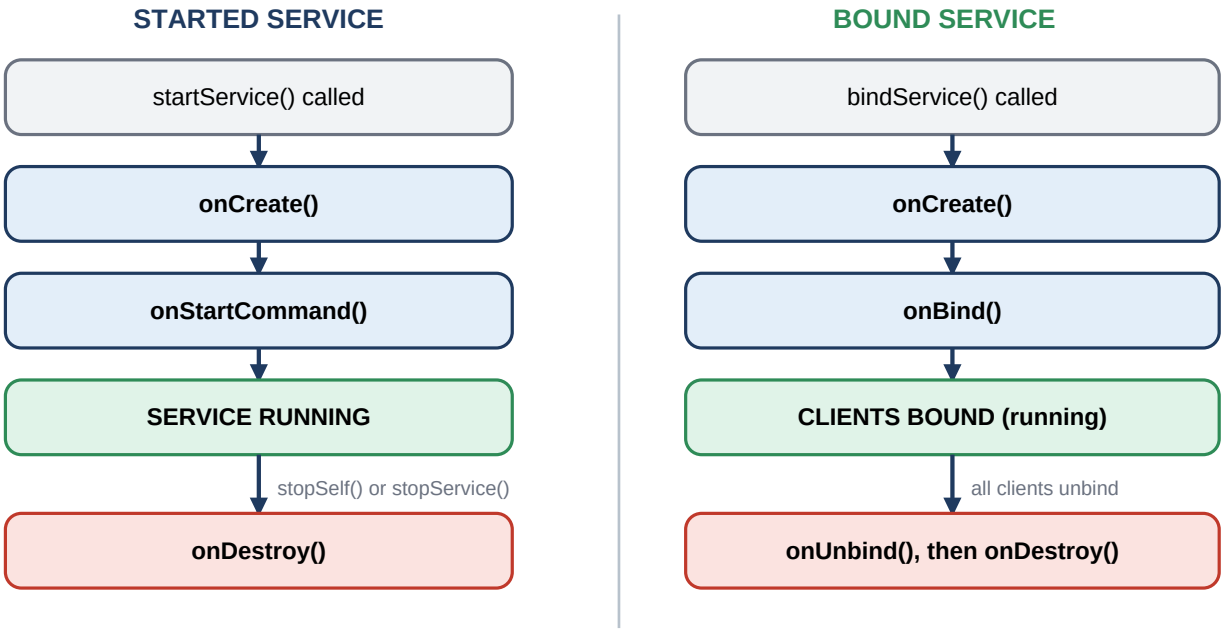


Figure 5: Life cycle of started and bound services

Callback	Purpose
onCreate()	Service pehli baar banne par ek baar call hota hai (one-time setup).
onStartCommand()	Har baar startService() call hone par; kaam yahan hota hai. START_STICKY, START_NOT_STICKY ya START_REDELIVER_INTENT return karta hai.
onBind()	bindService() se call hota hai. IBinder return karta hai; binding allowed na ho to null return karo.

Callback	Purpose
onUnbind()	Jab saare clients unbind ho jaayein.
onDestroy()	Jab service ki zaroorat nahi rehti; yahan cleanup karo.

MusicService.java (started service)

```
public class MusicService extends Service {
    private MediaPlayer player;

    @Override
    public void onCreate() {
        super.onCreate();
        player = MediaPlayer.create(this, R.raw.song);   // song.mp3 in res/raw
        player.setLooping(true);
    }

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        player.start();
        return START_STICKY;          // restart the service if the system kills it
    }

    @Override
    public IBinder onBind(Intent intent) {
        return null;                  // not a bound service
    }

    @Override
    public void onDestroy() {
        super.onDestroy();
        player.stop();
        player.release();
    }
}
```

Starting, stopping and declaring the service

```
// Start and stop from an activity (for example in button click listeners)
startService(new Intent(this, MusicService.class));
stopService(new Intent(this, MusicService.class));

// Declare the service in the manifest (inside <application>)
// <service android:name=".MusicService" android:exported="false" />
```

Modern rules jo jaanne chahiye

1) Android 8.0 (API 26) se background me app freely background services start nahi kar sakta. Background kaam ke liye **WorkManager** use karo, ya user ko pata hone wale lamba chalne wale kaam ke liye **foreground service** (notification ke saath). **2) IntentService deprecated hai**; WorkManager use karo. **3)** Service start ya bind karne ke liye hamesha **explicit intent** use karo (implicit intent exception deta hai). **4)** Android 14 (API 34) me foreground service ko **foreground service type** declare karna padta hai.

Bound service (outline)

```
// Bound service (outline)
public class LocalService extends Service {
    private final IBinder binder = new LocalBinder();
    public class LocalBinder extends Binder {
        LocalService getService() { return LocalService.this; }
    }
    @Override public IBinder onBind(Intent intent) { return binder; }
    public int getRandomNumber() { return new Random().nextInt(100); }
}

// In the activity
bindService(new Intent(this, LocalService.class), connection, Context.BIND_AUTO_CREATE);
// in onStop():  unbindService(connection);
```

Topic 6: Intents

Real-life example

Intent ek **request slip** hai. Tum likh sakte ho 'Ye slip Ramesh ko dena' (explicit) ya 'Jo koi phone repair kar sake, madad karo' (implicit). Phir system sahi insaan dhoondhta hai.

Definition

Intent ek **messaging object** hai jo doosre app component se koi action maangne ke liye use hota hai. Isse **activity start**, **service start** aur **broadcast deliver** hota hai.

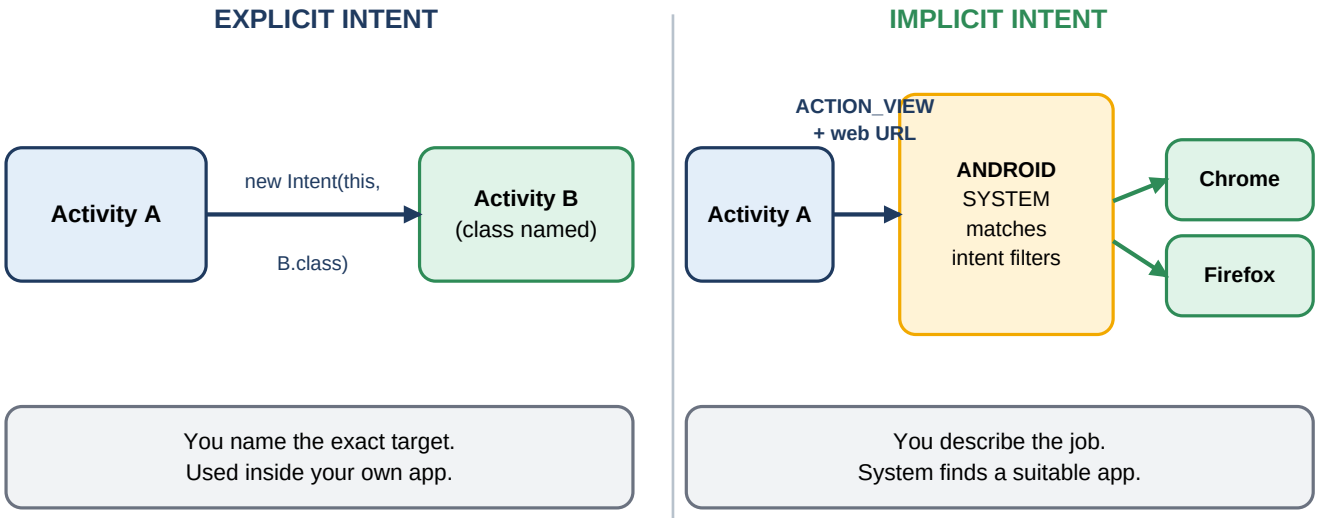


Figure 6: Explicit and implicit intents

Point	Explicit intent	Implicit intent
Target	Exact component class ka naam deta hai	Action batata hai; class ka naam nahi
Use	Apne hi app ke andar jaana	Doosre apps use karna (browser, dialer, share)
Kaun decide karta hai	Tum	Android system (intent filters match karta hai)
Example	new Intent(this, SecondActivity.class)	new Intent(Intent.ACTION_VIEW, uri)

Intent ke parts

Part	Matlab
Component name	Target class (sirf explicit intents)
Action	Kya karna hai: ACTION_VIEW, ACTION_SEND, ACTION_DIAL, ACTION_MAIN

Part	Matlab
Data	Data ka URI (tel:100, https://..., content://...) aur uska MIME type
Category	Component ke type ki extra information: CATEGORY_LAUNCHER, CATEGORY_BROWSABLE
Extras	Data le jaane wale key-value pairs (putExtra / getStringExtra)
Flags	Launch kaise karna hai, jaise FLAG_ACTIVITY_NEW_TASK

Intent examples

```
// 1. Explicit intent: open your own SecondActivity
Intent i = new Intent(MainActivity.this, SecondActivity.class);
startActivity(i);

// 2. Implicit: open a web page
Intent web = new Intent(Intent.ACTION_VIEW, Uri.parse("https://www.example.com"));
startActivity(web);

// 3. Implicit: open the dialer with a number (no permission needed)
Intent dial = new Intent(Intent.ACTION_DIAL, Uri.parse("tel:100"));
startActivity(dial);

// 4. Implicit: share text with any app
Intent send = new Intent(Intent.ACTION_SEND);
send.setType("text/plain");
send.putExtra(Intent.EXTRA_TEXT, "Hello from my app");
startActivity(Intent.createChooser(send, "Share via"));
```

- **ACTION_DIAL** sirf dialer kholta hai (permission nahi chahiye). **ACTION_CALL** seedha call lagata hai aur **CALL_PHONE** permission maangta hai.
- Agar koi app implicit intent handle na kar sake to app **ActivityNotFoundException** se crash hota hai. Pehle `intent.resolveActivity(getPackageManager())` se check karo (Android 11+ par `<queries>` entry chahiye ho sakti hai).

Activity se result wapas lena

Result from another activity

```
// Modern way (Activity Result API) in MainActivity
ActivityResultLauncher<Intent> launcher = registerForActivityResult(
    new ActivityResultContracts.StartActivityForResult(),
    result -> {
        if (result.getResultCode() == RESULT_OK && result.getData() != null) {
            String answer = result.getData().getStringExtra("answer");
        }
    });
launcher.launch(new Intent(this, SecondActivity.class));

// In SecondActivity: send the result back
Intent data = new Intent();
data.putExtra("answer", "42");
setResult(RESULT_OK, data);
finish();
```

Textbooks ka purana tareeka

Purani books `startActivityForResult(intent, 1)` aur `onActivityResult(int requestCode, int resultCode, Intent data)` override use karti hain. Ye abhi bhi chalta hai par upar dikhaye Activity Result API ke aage **deprecated** hai. Exam ke liye dono seekho.

Topic 7: Receiving and Broadcasting Intents

Real-life example

Broadcast railway station par **loudspeaker ki public announcement** jaisa hai: 'Train 12345 late hai'. Jo bhi sun raha hai (har registered receiver) wo sunta hai, aur har ek apna action decide karta hai.

Definition

Broadcast Receiver ek component hai jo system ya apps dwara bheje gaye **broadcast intents ko sunta** hai aur unpar react karta hai. **Broadcast** wo message hai jo **saare interested receivers** ko bheja jaata hai.

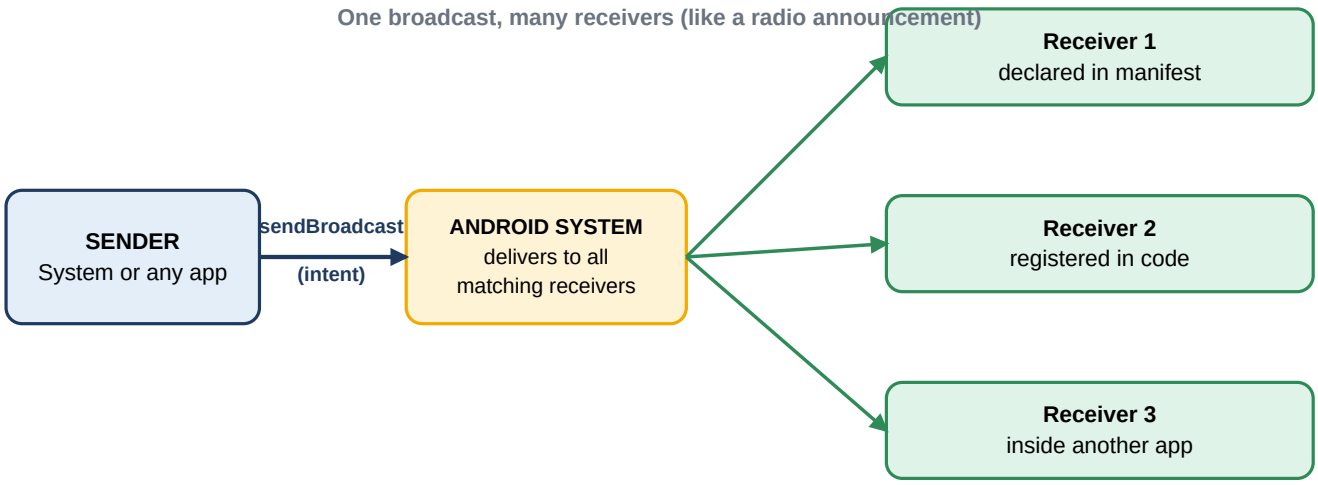


Figure 7: Broadcasting an intent to many receivers

Common system broadcasts

Action	Kab bheja jaata hai
ACTION_BOOT_COMPLETED	Device boot hona khatam karta hai (RECEIVE_BOOT_COMPLETED permission chahiye)
ACTION_BATTERY_LOW / ACTION_BATTERY_OKAY	Battery low / okay hoti hai
ACTION_POWER_CONNECTED / DISCONNECTED	Charger lagta / hatta hai
ACTION_AIRPLANE_MODE_CHANGED	Airplane mode badalta hai
ACTION_SCREEN_ON / SCREEN_OFF	Screen on / off hoti hai (sirf context-registered receivers ko milta hai)
SMS_RECEIVED (Telephony)	SMS aata hai (restricted; special permission chahiye)

Receiver register karne ke do tareeke

Point	Manifest-declared (static)	Context-registered (dynamic)
Kahan	AndroidManifest.xml me <receiver> tag	Code me registerReceiver()
Kab kaam karta hai	App na chal raha ho tab bhi (system start karta hai)	Sirf jab tak register karne wali activity/context zinda hai
Restriction	Android 8.0 se zyada tar implicit broadcasts yahan declare nahi ho sakte (kuch exceptions jaise BOOT_COMPLETED)	Restricted nahi; kaam ke baad unregister karna zaroori

MyReceiver.java

```
// 1. A receiver class
public class MyReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Toast.makeText(context, "Broadcast received: " + intent.getAction(),
                       Toast.LENGTH_SHORT).show();
    }
}
```

Registering a receiver (static and dynamic)

```
// 2a. Register in the manifest (static), e.g. to run after boot
// <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
// <receiver android:name=".MyReceiver" android:exported="true">
//     <intent-filter>
//         <action android:name="android.intent.action.BOOT_COMPLETED" />
//     </intent-filter>
// </receiver>

// 2b. Register in code (dynamic) inside an activity
private final MyReceiver receiver = new MyReceiver();

@Override protected void onStart() {
    super.onStart();
    IntentFilter filter = new IntentFilter(Intent.ACTION_BATTERY_LOW);
    ContextCompat.registerReceiver(this, receiver, filter,
            ContextCompat.RECEIVER_EXPORTED);       // listening to system broadcasts
}
@Override protected void onStop() {
    super.onStop();
    unregisterReceiver(receiver);                   // always unregister
}
```

Sending a broadcast

```
// 3. Broadcasting your own intent (inside your own app)
Intent intent = new Intent("com.example.myapp.MY_ACTION");
intent.setPackage(getPackageName());     // keep it inside this app
intent.putExtra("msg", "Hello receivers");
sendBroadcast(intent);

// sendOrderedBroadcast(intent, null) delivers to one receiver at a time, by priority.
```

- **onReceive()** main thread par chalta hai aur jaldi khatam hona chahiye (lagbhag 10 second). Wahan lamba kaam mat karo; uski jagah service ya WorkManager job start karo.
- **Normal broadcast** (sendBroadcast) saare receivers ko ek saath jaata hai. **Ordered broadcast** (sendOrderedBroadcast) priority order me ek ek receiver ko jaata hai, aur receiver use aage bhej ya abort kar sakta hai.
- **Android 14 (API 34):** agar tum use target karte ho aur non-system broadcasts ke liye receiver code me register karte ho, to **RECEIVER_EXPORTED** ya **RECEIVER_NOT_EXPORTED** specify karna zaroori hai. ContextCompat.registerReceiver() use karo taaki flag purane versions par bhi chale.
- **Security:** broadcast doosre apps bhi padh sakte hain. Isme secret kabhi mat daalo; use `setPackage()` ya permission se restrict karo.

Topic 8: Android Manifest File and its Common Settings

Manifest Unit I me dekha tha. Yahan wo **settings hain jo sabse zyada use hoti hain**. Har activity, service, receiver aur provider yahan declare hona chahiye, aur manifest me permissions aur app-wide settings bhi hoti hain.

Element	Common settings (attributes)
<uses-permission>	android:name = permission, jaise android.permission.INTERNET, CAMERA
<uses-feature>	android:name (jaise android.hardware.camera), **android:required** (true ya false)
<uses-sdk>	minSdkVersion, targetSdkVersion (purane projects; ab build.gradle me)
<supports-screens>	App kaunsi screen sizes support karta hai (small, normal, large)
<application>	**icon, roundIcon, label, theme**, allowBackup, supportsRtl, name (custom Application class), usesCleartextTraffic (plain HTTP allow)
<activity>	**name, label, icon, theme, exported, launchMode, screenOrientation** (portrait / landscape), configChanges, windowSoftInputMode, parentActivityName
<service>	name, exported, enabled, permission, foregroundServiceType, process
<receiver>	name, exported, enabled, permission, aur ek <intent-filter>
<provider>	name, **authorities**, exported, grantUriPermissions
<intent-filter>	Component ka child: <action>, <category>, <data>
<permission>	Custom permission define karta hai: name, protectionLevel
<meta-data>	Component ke liye extra name-value data (jaise API key)
<queries>	Wo doosre apps jinhe tumhara app dekhna chahta hai (Android 11+ package visibility)

A manifest with the most common settings

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapp">

    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.CAMERA" />
    <uses-feature android:name="android.hardware.camera" android:required="false" />

    <application
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/Theme.MyApp">

        <activity android:name=".MainActivity" android:exported="true"
            android:screenOrientation="portrait">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity android:name=".SecondActivity" android:exported="false"
            android:parentActivityName=".MainActivity" />

        <service android:name=".MusicService" android:exported="false" />

        <receiver android:name=".MyReceiver" android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.BOOT_COMPLETED" />
            </intent-filter>
        </receiver>

    </application>
</manifest>
```

- **Sirf ek activity** me MAIN + LAUNCHER filter hota hai: wahi jo app icon dabane par khulti hai.
- **android:exported:** Android 12 (API 31) ya usse upar target karne par intent filter wale har component ke liye explicitly set karna zaroori hai. Jab tak doosre apps ko component tak pahunchna na ho, **false** rakho.
- **screenOrientation:** screen ko portrait ya landscape me lock karta hai. **configChanges:** Android ko batata hai ki activity kuch changes (jaise rotation) khud handle karti hai, isliye wo dobara nahi banti.
- **usesCleartextTraffic:** Android 9 (API 28) ya usse upar target karne wale apps plain HTTP ko by default block karte hain; HTTPS use karo, ya cleartext sirf testing ke liye allow karo.

Topic 9: Using Intent Filter

Real-life example

Intent filter **dukaan ke bahar ke board** jaisa hai: 'Yahan mobile repair hota hai'. Wo service dhoondhne wale log dukaan tak pahunch jaate hain. Isi tarah intent filter Android ko batata hai ki component kaunsi requests handle kar sakta hai.

Definition

Intent filter manifest ka ek expression hai jo **batata hai ki component kaunse types ke intents receive kar sakta hai**. System implicit intent ko saare apps ke filters se compare karke matching components chunta hai.

Element	Matlab	Example
<action>	Component kaunsa action kar sakta hai	android.intent.action.VIEW
<category>	Component ka type ya use karne ka tareeka	DEFAULT, LAUNCHER, BROWSABLE
<data>	Data ka type: scheme, host, port, path, mimeType	scheme=https, host=www.example.com

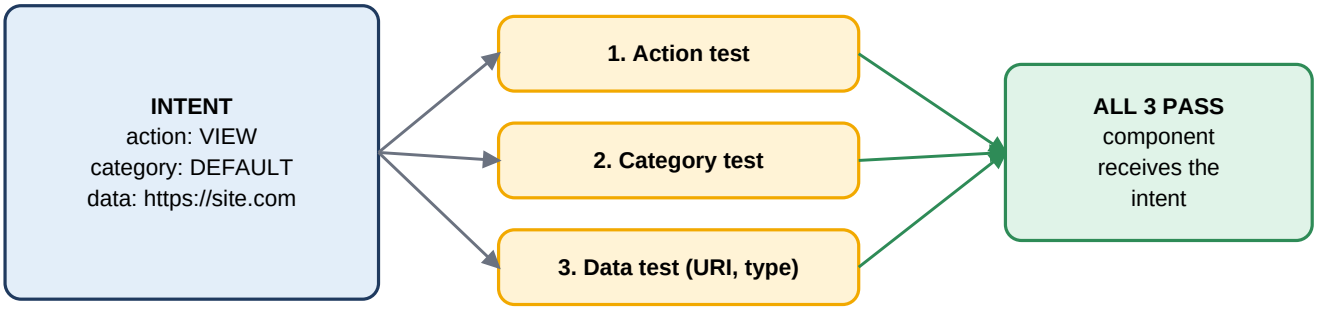


Figure 8: Matching an intent against an intent filter

Matching rules

- Intent ko **teeno tests** pass karne padte hain: action, category aur data.
- **Action test:** intent ka action filter me listed actions me se kisi ek se match karna chahiye.
- **Category test:** intent ki har category filter me hona zaroori hai. startActivity() se bheje implicit intents me system **CATEGORY_DEFAULT** jodta hai, isliye implicit intents chahne wali activity me ye hona chahiye.
- **Data test:** intent ka URI aur MIME type filter ke data element se match karna chahiye.
- Kai apps match karein to user ko **chooser** dialog dikhta hai.

Common intent filters

```
<!-- 1. Launcher activity: the first screen when the app icon is tapped -->
<intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
</intent-filter>

<!-- 2. Open web links of your site in your app (deep link) -->
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="https" android:host="www.example.com" />
</intent-filter>

<!-- 3. Receive text shared from other apps -->
<intent-filter>
    <action android:name="android.intent.action.SEND" />
    <category android:name="android.intent.category.DEFAULT" />
    <data android:mimeType="text/plain" />
</intent-filter>
```

- **Services:** hamesha **explicit** intent se start ya bind karo. bindService() ke liye implicit intent exception deta hai, aur security ke liye service ko intent filter par depend nahi karna chahiye.
- **Broadcast receivers** intent filters se batate hain ki unhe kaunse broadcasts chahiye (static receivers manifest me, dynamic wale IntentFilter object se).
- **Security:** intent filter security boundary nahi hai. Isse mile data par bharosa mat karo; use validate karo.

Topic 10: Permissions

Real-life example

Android ek **secured office building** jaisa hai. Apps visitors hain. Server room (camera, contacts, location) me jaane ke liye visitor ko **pass** chahiye. Kuch pass gate par mil jaate hain (normal), par sensitive rooms ke liye building owner (user) se approval maangti hai.

Definition

Permission ek **restriction** hai jo sensitive data (contacts, SMS) aur features (camera, location) ko protect karti hai. App ko apni zaroori permissions manifest me **declare** karni padti hain, aur dangerous permissions ke liye **run time par user se bhi poochna** padta hai.

Permissions ke types (protection levels)

Type	Kaise milti hai	Examples
Normal	Kam risk. Manifest me declare hone par **install par apne aap** mil jaati hai.	INTERNET, ACCESS_NETWORK_STATE, VIBRATE, SET_ALARM
Dangerous (runtime)	Private data ya sensitive features. App ko **run time par user se poochna** padta hai (Android 6.0, API 23 aur upar).	CAMERA, RECORD_AUDIO, READ_CONTACTS, ACCESS_FINE_LOCATION, SEND_SMS, CALL_PHONE, POST_NOTIFICATIONS (Android 13)
Signature	Sirf unhi apps ko milti hai jo permission define karne wale app ke same certificate se signed hon.	Apne hi apps ke beech custom permissions

Type	Kaise milti hai	Examples
Special	User special Settings screen se deta hai, normal dialog se nahi.	SYSTEM_ALERT_WINDOW, WRITE_SETTINGS, MANAGE_EXTERNAL_STORAGE

- **Android 6.0 se pehle** saari permissions install ke time dikhti thi aur ek saath accept hoti thi. **Android 6.0 se** dangerous permissions feature use hone par maangi jaati hain, aur user baad me Settings se unhe **revoke** kar sakta hai.
- Dangerous permissions **permission groups** me baanti hoti hain jaise CONTACTS, LOCATION, CAMERA, MICROPHONE, PHONE aur SMS.
- **Android 13 (API 33):** notifications dikhane ke liye runtime permission **POST_NOTIFICATIONS** chahiye, aur media access ke liye READ_EXTERNAL_STORAGE ki jagah READ_MEDIA_IMAGES / READ_MEDIA_VIDEO / READ_MEDIA_AUDIO use hoti hain.

Dangerous permission run time par maangna (steps)

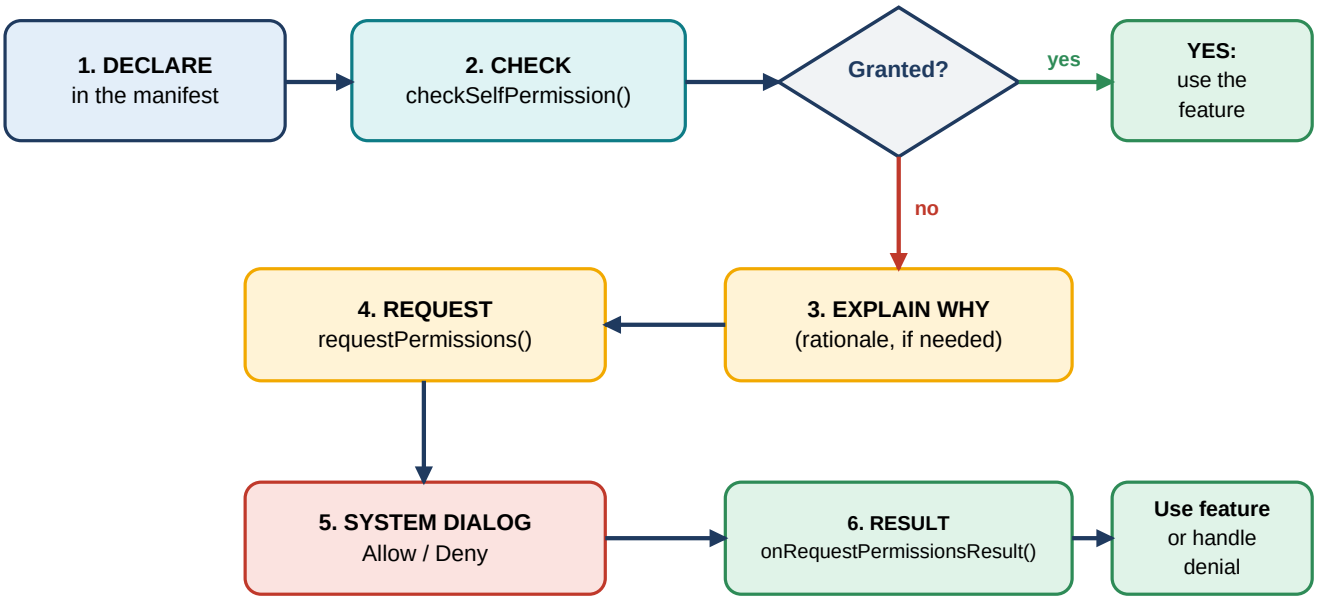


Figure 9: Runtime permission flow

Runtime permission: classic way

```
// Step 1: declare in AndroidManifest.xml
// <uses-permission android:name="android.permission.CAMERA" />

// Step 2 and 3: check, and request if not granted
private static final int CAMERA_CODE = 100;

private void startCameraFeature() {
    if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
            != PackageManager.PERMISSION_GRANTED) {
        ActivityCompat.requestPermissions(this,
                new String[]{Manifest.permission.CAMERA}, CAMERA_CODE);
    } else {
        openCamera();                          // already granted
    }
}

// Step 4: handle the user's answer
@Override
public void onRequestPermissionsResult(int requestCode,
        @NonNull String[] permissions, @NonNull int[] grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);
    if (requestCode == CAMERA_CODE) {
        if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
            openCamera();
        } else {
            Toast.makeText(this, "Camera permission denied", Toast.LENGTH_SHORT).show();
        }
    }
}
```

Runtime permission: modern way

```
// Modern way: Activity Result API (a launcher replaces onRequestPermissionsResult)
private final ActivityResultLauncher<String> cameraPermission =
    registerForActivityResult(new ActivityResultContracts.RequestPermission(),
        isGranted -> {
            if (isGranted) openCamera();
            else Toast.makeText(this, "Camera permission denied", Toast.LENGTH_SHORT).show();
        });

// when the user taps a button
cameraPermission.launch(Manifest.permission.CAMERA);
```

- **shouldShowRequestPermissionRationale()** tab true deta hai jab user pehle deny kar chuka ho; dobara poochne se pehle batao ki permission kyun chahiye.
- User deny karke 'Don't ask again' chune (ya baar baar deny kare) to dialog nahi aata; user ko **Settings** tak guide karo.
- **Best practice:** sirf zaroori permissions maango, **zaroorat ke time** maango (app start par nahi), kyun chahiye batao, aur deny hone par bhi app (kam features ke saath) chalna chahiye.

Custom permissions

Defining and using a custom permission

```
<!-- Define a permission in your app -->
<permission android:name="com.example.myapp.permission.MY_PERM"
            android:protectionLevel="signature" />

<!-- Protect a component with it: only apps holding the permission can start it -->
<service android:name=".MyService" android:exported="true"
         android:permission="com.example.myapp.permission.MY_PERM" />
```

PART B: Important Questions aur Model Answers (Hinglish)

Priority: **Very Likely** = BTEUP me sabse zyada aata hai. **Likely** = achha chance. **Possible** = aa sakta hai. Har answer exam format me hai: **Definition, Points, Diagram/Table/Code, Conclusion**.

Q1. Important Android terminologies explain karo. — LIKELY

Definition / Parichay: Android me kuch basic terms hain jo app ke building blocks aur tools ko describe karte hain. Android development samajhne ke liye inhe jaanna zaroori hai.

Points

- **Activity:** UI wali ek screen. **Service:** bina UI ka background kaam.
- **Broadcast Receiver:** system ya app messages sunta hai. **Content Provider:** apps ke beech data share karta hai.
- **Intent:** message jo component start karta hai ya data le jaata hai. **Intent Filter:** batata hai ki component kaunse intents handle karega.
- **View / ViewGroup / Layout:** UI element, views ka container, aur unhe arrange karne wali XML. **Fragment:** screen ka reusable hissa.
- **Context:** app environment ka handle. **Manifest:** app ka ID card. **Resources / R class:** non-code files aur unki IDs.
- **APK / AAB, AVD, ADB, Logcat, Gradle, Bundle, Toast, Notification, Permission, Back stack** anya common terms hain.

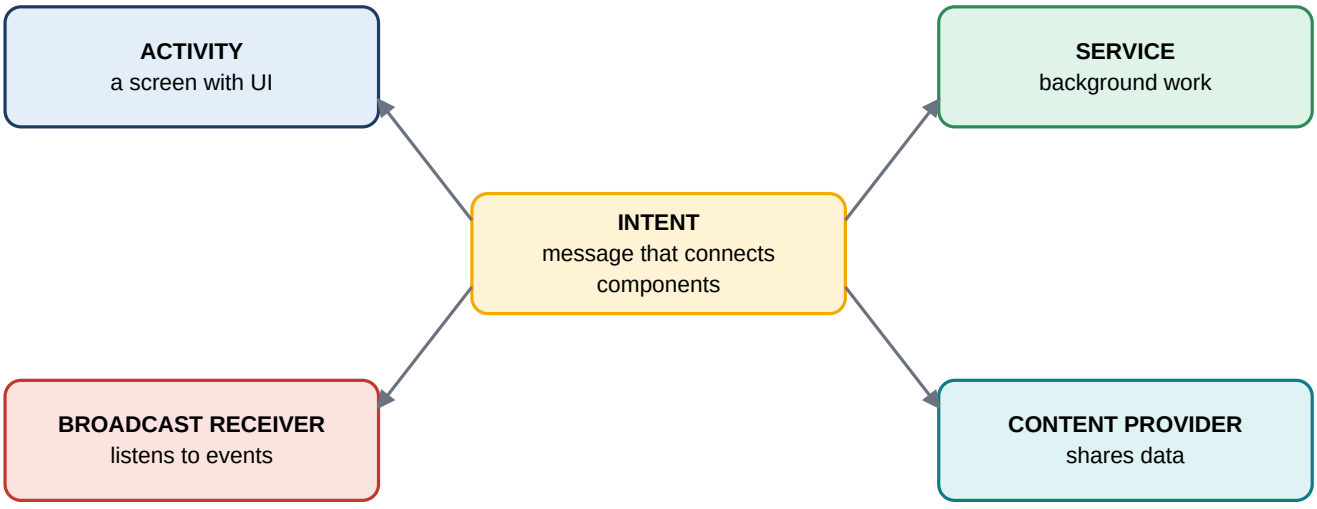


Figure: Building blocks of an Android app

Conclusion: Ye terms Android development ki vocabulary hain aur poore syllabus me aate hain.

Q2. Android me Context kya hai? Application Context aur Activity Context explain karo.

LIKELY

Definition / Parichay: Context ek abstract class hai jo application environment ki global information, jaise resources, assets, databases aur system services, tak access deti hai aur code ko components launch karne deti hai.

Points

- **Application context** `getApplicationContext()` se milta hai. Ye poore application ki life se juda hai aur long-lived objects me rakhna safe hai.
- **Activity context** `this` ya `ActivityName.this` se milta hai. Ye utni der rehta hai jitni wo activity, aur uska theme saath rakhta hai. Dialogs aur themed layouts ko ye chahiye.
- **Memory leak:** long-lived object me activity context rakhne se activity free nahi ho paati.
- **Application class:** `android.app.Application` global state rakhti hai; ye kisi bhi doosre component se pehle banti hai.

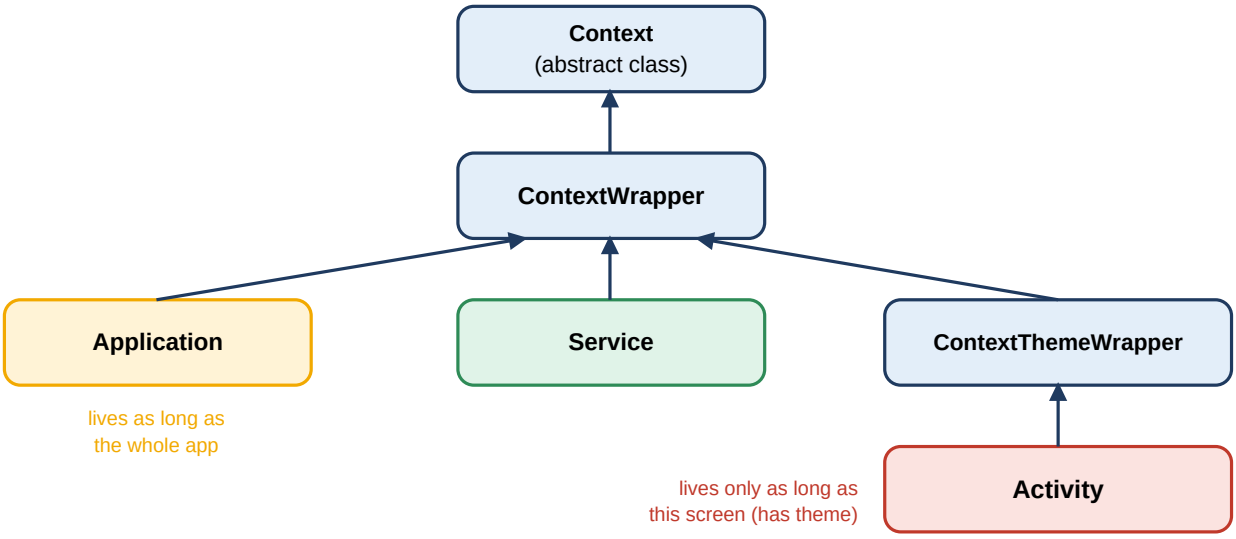


Figure: Context hierarchy

Point	Application context	Activity context
Lifetime	Poora app	Sirf wahi activity
Dialog dikhana	Nahi	Haan
Singleton me safe	Haan	Nahi (leak risk)

Conclusion: Sahi context chunne se crashes aur memory leaks se bacha ja sakta hai.

Q3. Activity kya hai? Activity life cycle diagram ke saath explain karo. VERY LIKELY

Definition / Parichay: Activity ek application component hai jo ek screen deta hai jisse users interact karte hain. System har activity ko life-cycle callback methods se manage karta hai jo activity ke bante, dikhte, chhupte aur destroy hote samay chalte hain.

Points

- **onCreate():** activity pehli baar banne par call hota hai; layout set karo aur data initialise karo.
- **onStart():** activity visible hoti hai. **onResume():** wo foreground me hai aur user interact kar sakta hai (running state).
- **onPause():** doosri activity saamne aa rahi hai; halka data save karo aur resources chhodo.
- **onStop():** activity ab visible nahi. **onRestart():** wo stopped state se wapas aati hai, uske baad onStart().
- **onDestroy():** activity destroy hone se pehle final cleanup (finish() ya system kill kare).
- **Lifetimes:** entire (onCreate se onDestroy), visible (onStart se onStop), foreground (onResume se onPause).

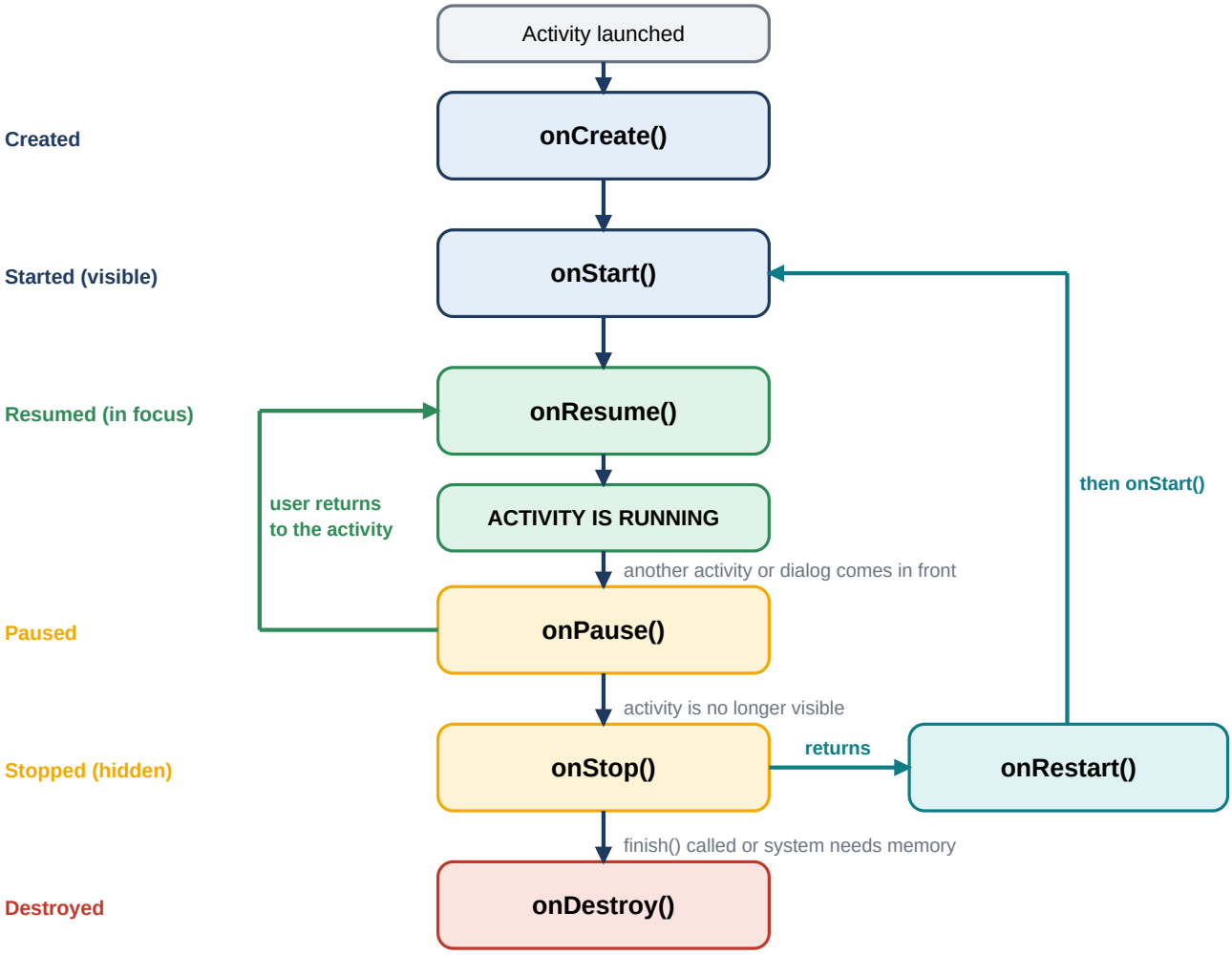


Figure: Activity life cycle

Scenario	Methods called
Pehli baar launch	onCreate, onStart, onResume
Home button	onPause, onStop
App me wapas	onRestart, onStart, onResume

Scenario	Methods called
Back button	onPause, onStop, onDestroy

Conclusion: Life cycle samajhne se developers sahi samay par data save kar sakte hain aur resources release kar sakte hain.

Q4. Doosri activity start karne aur usse data pass karne ka program likho. Activities kaise declare hoti hain? VERY LIKELY

Definition / Parichay: Activities Intents ke zariye communicate karti hain. Explicit intent target activity ka naam deta hai, aur extras data ko key-value pairs me le jaate hain.

Points

- Current context aur target class ke saath **Intent** banao, `putExtra()` se data jodo, aur `startActivity()` call karo.
- Target activity me data `getIntent().getStringExtra("key")` se padho.
- Har activity ko manifest me <activity> element se **declare** karna zaroori hai, nahi to app ActivityNotFoundException se crash hota hai.
- `finish()` current activity ko band karke pichhli activity par wapas bhejta hai.

Passing data between two activities

```
// MainActivity.java
Button btn = findViewById(R.id.btnNext);
btn.setOnClickListener(v -> {
    Intent intent = new Intent(MainActivity.this, SecondActivity.class);
    intent.putExtra("name", "Amrit");
    startActivity(intent);
});

// SecondActivity.java (inside onCreate)
String name = getIntent().getStringExtra("name");
TextView tv = findViewById(R.id.tvName);
tv.setText("Hello " + name);

// AndroidManifest.xml
// <activity android:name=".SecondActivity" android:exported="false" />
```

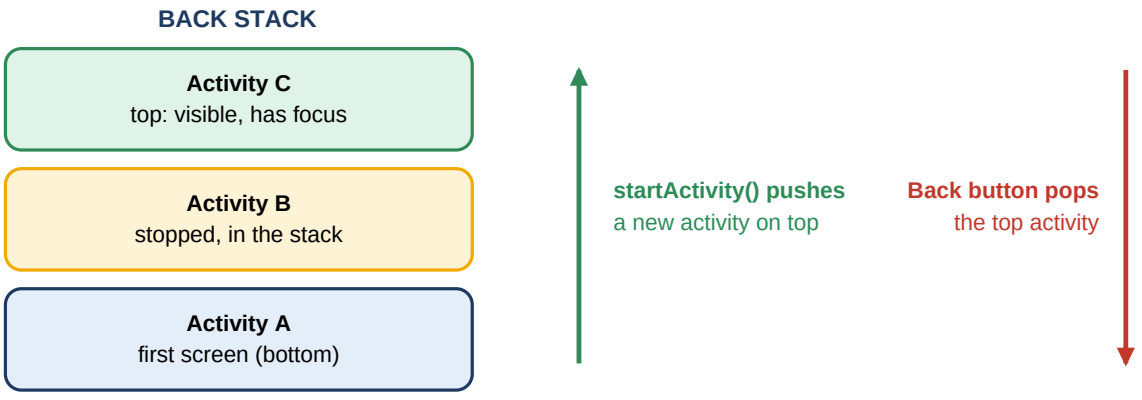


Figure: Back stack

Conclusion: Extras ke saath explicit intent screens ke beech jaane aur data share karne ka standard tarika hai.

Q5. Service kya hai? Uske types aur life cycle explain karo. VERY LIKELY

Definition / Parichay: Service ek application component hai jo background me long-running operations karta hai aur user interface nahi deta.

Points

- **Foreground service:** user ko dikhne wala kaam karti hai aur notification dikhana zaroori (jaise music player).
- **Background service:** user ko na dikhne wala kaam; Android 8.0 se bahut restricted, isliye WorkManager prefer hota hai.
- **Bound service:** component bindService() se bind hota hai; client-server jaisa kaam karti hai aur clients bound hone tak hi chalti hai.
- **Started service life cycle:** startService() se onCreate(), phir onStartCommand(), service chalti hai, aur stopSelf() ya stopService() se onDestroy().
- **Bound service life cycle:** bindService() se onCreate(), phir onBind(); saare clients unbind hone par onUnbind() aur onDestroy() call hote hain.
- Service by default main thread par chalti hai; ANR se bachne ke liye heavy kaam background thread par karo. Isse manifest me declare karna zaroori hai.

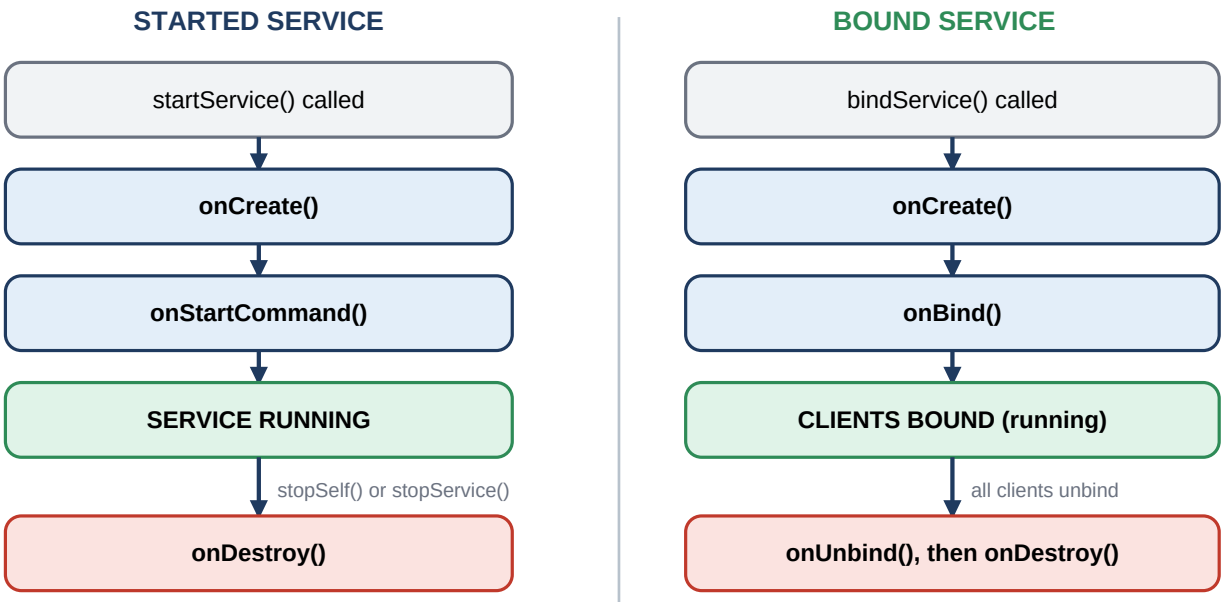


Figure: Life cycle of started and bound services

Conclusion: Services app ko aisa kaam karne deti hain jo user ke screen chhodne ke baad bhi chalta rahe.

Q6. Service banane aur use karne ka program likho (music player example). LIKELY

Definition / Parichay: Started service background me music bajati hai chahe user activity chhod de. Hum service class banate hain, use manifest me declare karte hain aur activity se start aur stop karte hain.

Points

- **Step 1:** `Service` class extend karo aur onCreate(), onStartCommand(), onBind() aur onDestroy() override karo.
- **Step 2:** audio file `res/raw` me rakho aur MediaPlayer banao.
- **Step 3:** service ko manifest me declare karo.
- **Step 4:** button clicks se startService() se start aur stopService() se stop karo.

MusicService.java and usage

```
public class MusicService extends Service {
    private MediaPlayer player;

    @Override public void onCreate() {
        super.onCreate();
        player = MediaPlayer.create(this, R.raw.song);
        player.setLooping(true);
    }
    @Override public int onStartCommand(Intent intent, int flags, int startId) {
        player.start();
        return START_STICKY;
    }
    @Override public IBinder onBind(Intent intent) { return null; }
    @Override public void onDestroy() {
        super.onDestroy();
        player.stop();
        player.release();
    }
}

// Activity: start / stop
startService(new Intent(this, MusicService.class));
stopService(new Intent(this, MusicService.class));

// Manifest:  <service android:name=".MusicService" android:exported="false" />
```

Conclusion: Service tab tak gaana bajati rehti hai jab tak stopService() call na ho, screen se independent.

Q7. Intents kya hain? Explicit aur implicit intents examples ke saath explain karo. VERY LIKELY

Definition / Parichay: Intent ek messaging object hai jo doosre application component se action maangne ke liye use hota hai. Isse activities aur services start hoti hain aur broadcasts deliver hote hain.

Points

- **Explicit intent:** exact target component class ka naam deta hai. Apne hi app ke andar use hota hai. Example: `new Intent(this, SecondActivity.class)`.
- **Implicit intent:** action aur data batata hai; system intent filters se suitable app dhoondhta hai. Example: `new Intent(Intent.ACTION_VIEW, Uri.parse("https://..."))`.
- **Intent ke parts:** component name, action, data, category, extras aur flags.
- Data `putExtra()` se pass hota hai aur `getStringExtra()` se padha jaata hai; result Activity Result API (ya purane startActivityForResult()) se wapas aata hai.

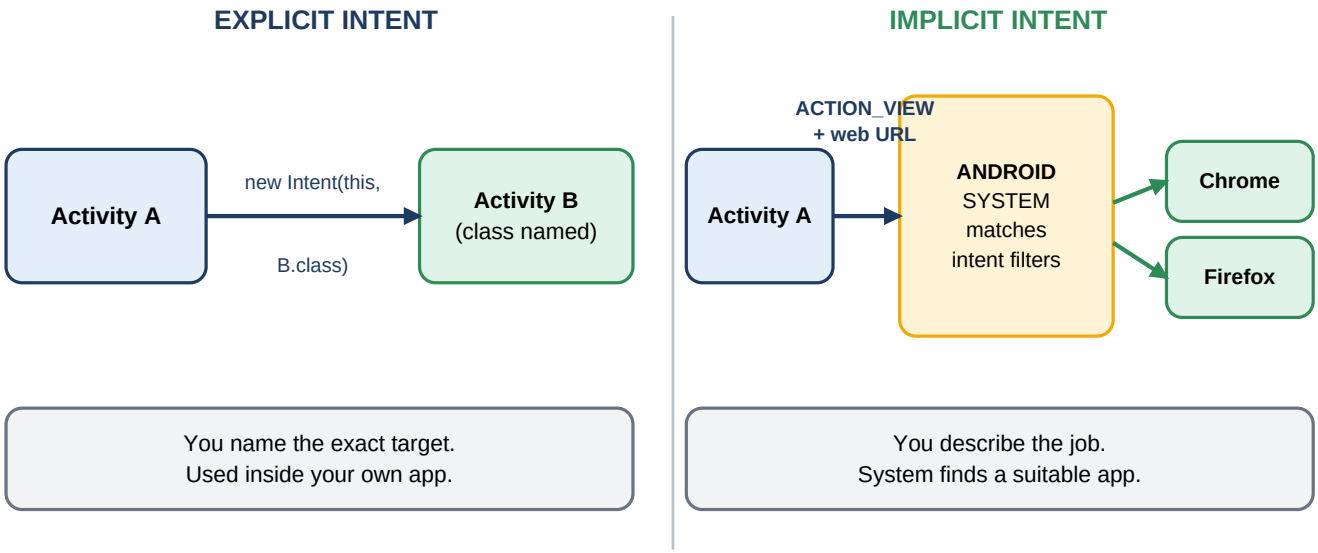


Figure: Explicit and implicit intents

Implicit intent examples

```
Intent dial = new Intent(Intent.ACTION_DIAL, Uri.parse("tel:100"));
startActivity(dial);

Intent send = new Intent(Intent.ACTION_SEND);
send.setType("text/plain");
send.putExtra(Intent.EXTRA_TEXT, "Hello");
startActivity(Intent.createChooser(send, "Share via"));
```

Conclusion: Intents Android components ko loosely coupled banate hain aur apps ko ek doosre ke features reuse karne dete hain.

Q8. Broadcast Receiver se intents receive aur broadcast karna explain karo.

VERY LIKELY

Definition / Parichay: Broadcast Receiver ek component hai jo system ya apps ke bheje broadcast intents ko sunta hai aur unpar react karta hai. Broadcast wo message hai jo saare interested receivers ko deliver hota hai.

Points

- **Receiver banao:** `BroadcastReceiver` extend karo aur `onReceive(Context, Intent)` override karo.
- **Static register** manifest me <receiver> aur <intent-filter> se; app na chal raha ho tab bhi kaam karta hai. Android 8.0 se zyada tar implicit broadcasts is tarah declare nahi ho sakte (BOOT_COMPLETED exception hai).
- **Dynamic register** code me registerReceiver() aur IntentFilter se; onStop()/onDestroy() me unregister karo. Android 14 par RECEIVER_EXPORTED ya RECEIVER_NOT_EXPORTED specify karo.
- **Broadcast bhejna:** custom action ke saath Intent banao aur `sendBroadcast()` (saare receivers) ya `sendOrderedBroadcast()` (priority ke hisab se ek ek) call karo.
- **System broadcasts:** BOOT_COMPLETED, BATTERY_LOW, POWER_CONNECTED, AIRPLANE_MODE_CHANGED, SCREEN_ON/OFF.

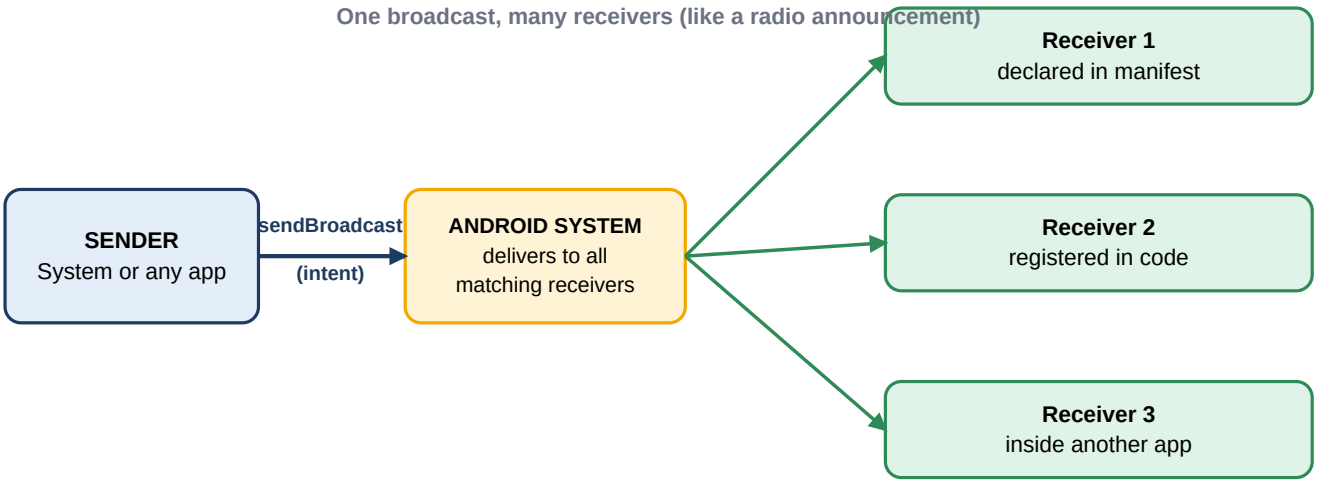


Figure: Broadcast to many receivers

Receiver and broadcast

```
public class MyReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Toast.makeText(context, "Received: " + intent.getAction(),
                       Toast.LENGTH_SHORT).show();
    }
}

// Send a broadcast from an activity
Intent i = new Intent("com.example.myapp.MY_ACTION");
i.setPackage(getPackageName());
sendBroadcast(i);
```

Conclusion: Broadcasts components ko events par react karne dete hain, bina ye jaane ki bheja kisne.

Q9. Android manifest file aur uski common settings explain karo. VERY LIKELY

Definition / Parichay: AndroidManifest.xml har app ke root me ek zaroori XML file hai jo Android system ko app ke baare me batati hai: package, components, permissions aur requirements. System koi bhi code chalane se pehle ise padhta hai.

Points

- **<uses-permission>:** app ko chahiye permissions (INTERNET, CAMERA). **<uses-feature>:** zaroori hardware, android:required ke saath.
- **<application>:** icon, label, theme, allowBackup, name; ye saare components ko contain karta hai.
- **<activity>:** name, label, icon, theme, exported, launchMode, screenOrientation, parentActivityName.
- **<service>, <receiver>, <provider>:** service, broadcast receiver aur content provider declare karte hain (name, exported, permission, authorities).
- **<intent-filter>:** action, category aur data; MAIN + LAUNCHER launcher activity batata hai.
- **android:exported** Android 12 ya usse upar target karne par intent filter wale components ke liye set karna zaroori hai.

Manifest example

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapp">
    <uses-permission android:name="android.permission.INTERNET" />
    <application android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name" android:theme="@style/Theme.MyApp">
        <activity android:name=".MainActivity" android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <service android:name=".MusicService" android:exported="false" />
    </application>
</manifest>
```

Conclusion: Manifest app ke saare parts ko jodta hai aur system ko batata hai ki app kya hai aur use kya chahiye.

Q10. Intent Filter kya hai? Examples ke saath explain karo. VERY LIKELY

Definition / Parichay: Intent filter manifest ka ek expression hai jo batata hai ki component kaunse types ke intents receive kar sakta hai. System implicit intent ko saare apps ke filters se compare karke matching components dhoondhta hai.

Points

- **<action>:** jo action kiya ja sakta hai (VIEW, SEND, MAIN).
- **<category>:** component kaise use hota hai (DEFAULT, LAUNCHER, BROWSABLE).
- **<data>:** data ka type: scheme, host, port, path, mimeType.
- **Matching:** intent ko action test, category test aur data test pass karne padte hain. Kai apps match karein to chooser dialog dikhta hai.
- **Uses:** launcher activity (MAIN + LAUNCHER), links kholna (VIEW + BROWSABLE), shared text receive karna (SEND), aur broadcasts ke liye receivers.

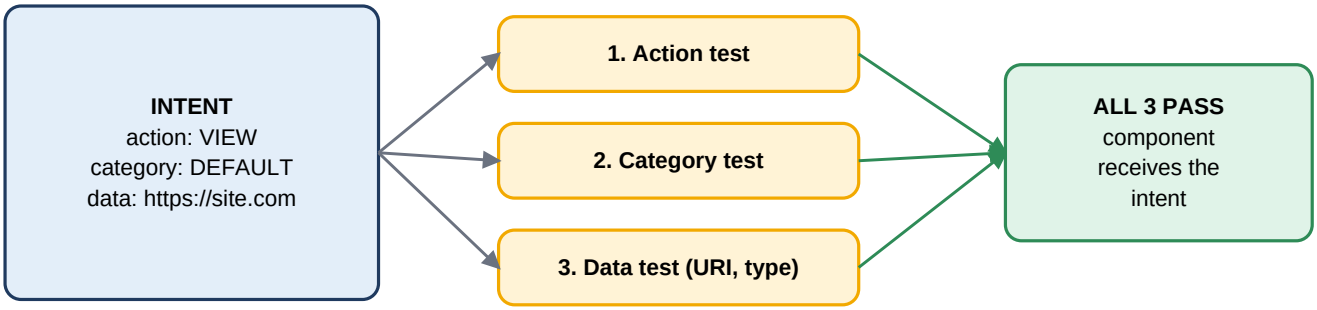


Figure: Intent matching

Receiving shared text

```
<activity android:name=".ShareActivity" android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.SEND" />
        <category android:name="android.intent.category.DEFAULT" />
        <data android:mimeType="text/plain" />
    </intent-filter>
</activity>
```

Conclusion: Intent filters apps ko apne features doosre apps ko offer karne dete hain, bina unhe pehle se jaane.

Q11. Android permissions kya hain? Unke types explain karo. VERY LIKELY

Definition / Parichay: Permission ek restriction hai jo device ke sensitive data aur features ko protect karti hai. App ko apni zaroori permissions manifest me declare karni padti hain, aur dangerous permissions ke liye run time par user se bhi poochna padta hai.

Points

- **Normal permissions:** kam risk; install par apne aap milti hain. Examples: INTERNET, ACCESS_NETWORK_STATE, VIBRATE.
- **Dangerous (runtime) permissions:** private data ya sensitive features; user ko run time par approve karna padta hai (Android 6.0+). Examples: CAMERA, RECORD_AUDIO, READ_CONTACTS, ACCESS_FINE_LOCATION, SEND_SMS, POST_NOTIFICATIONS.
- **Signature permissions:** sirf same certificate se signed apps ko milti hain.
- **Special permissions:** special Settings screen se milti hain, jaise SYSTEM_ALERT_WINDOW.
- **Declaration:** manifest me `<uses-permission android:name="android.permission.CAMERA" />`. Users baad me Settings se dangerous permissions revoke kar sakte hain.

Type	Kaise milti hai	Examples
Normal	Install par, apne aap	INTERNET, VIBRATE
Dangerous	User run time par approve	CAMERA, READ_CONTACTS
Signature	Same signing certificate	Custom permission
Special	Settings screen se	SYSTEM_ALERT_WINDOW

Conclusion: Permission model users ko unke private data par control deta hai aur apps ko zyada bharosemand banata hai.

Q12. Dangerous permission ko run time par maangne ka tareeka code ke saath explain karo. LIKELY

Definition / Parichay: Android 6.0 (API 23) se dangerous permissions ko sirf manifest me declare karna kaafi nahi, app chalte samay maangna bhi padta hai.

Points

- **Step 1:** permission ko manifest me <uses-permission> se declare karo.
- **Step 2:** `ContextCompat.checkSelfPermission()` se check karo.
- **Step 3:** granted na ho to optionally kyun chahiye batao (shouldShowRequestPermissionRationale) aur `ActivityCompat.requestPermissions()` call karo.
- **Step 4:** user ka jawab `onRequestPermissionsResult()` (ya Activity Result API callback) me handle karo.

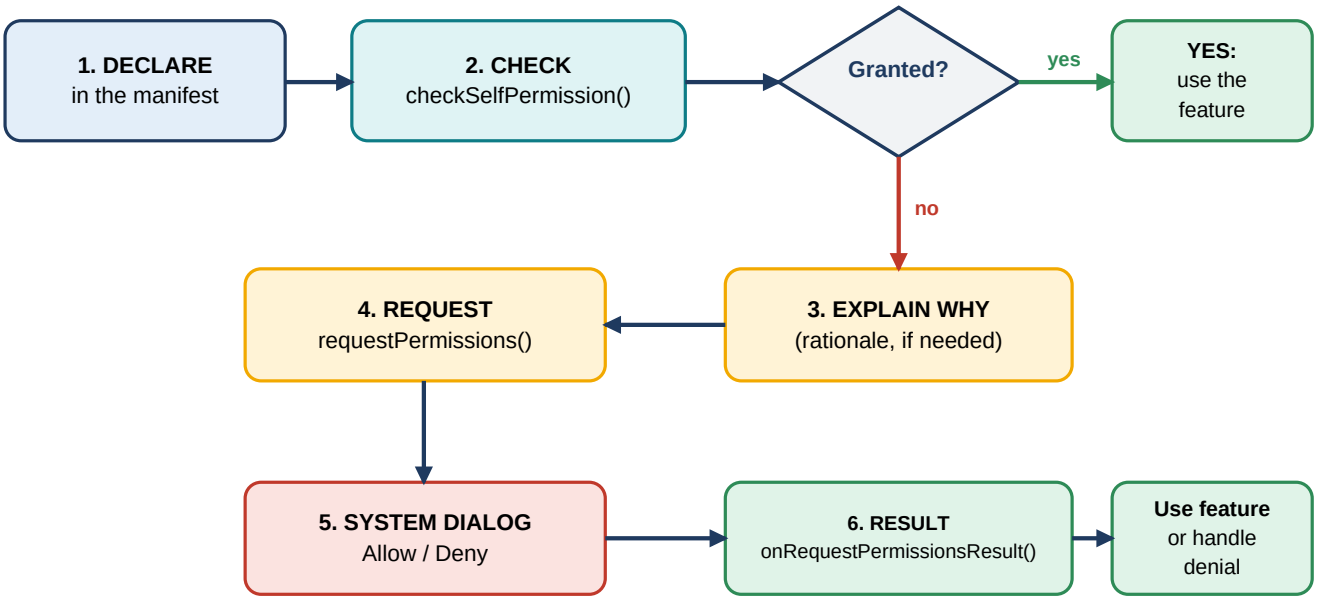


Figure: Runtime permission flow

Runtime permission code

```
if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
        != PackageManager.PERMISSION_GRANTED) {
    ActivityCompat.requestPermissions(this,
            new String[]{Manifest.permission.CAMERA}, 100);
} else {
    openCamera();
}

@Override
public void onRequestPermissionsResult(int code, @NonNull String[] perms,
                                       @NonNull int[] results) {
    super.onRequestPermissionsResult(code, perms, results);
    if (code == 100 && results.length > 0
            && results[0] == PackageManager.PERMISSION_GRANTED) {
        openCamera();
    }
}
```

Conclusion: Permission sirf zaroorat par aur saaf wajah ke saath maangne se user ka bharosa banta hai.

Q13. Difference likho: (a) Activity aur Service, (b) Started aur Bound service, (c) Explicit aur Implicit intent.

POSSIBLE

Definition / Parichay: Ye jodiyan aksar confuse hoti hain. Neeche ki tables inhe compare karti hain.

Points

- Har table dono items ko point by point compare karti hai.

(a) Point	Activity	Service
User interface	UI hai (ek screen)	UI nahi
Purpose	User se interact karna	Background kaam
Life cycle	onCreate, onStart, onResume, onPause, onStop, onDestroy	onCreate, onStartCommand / onBind, onDestroy
Example	Login screen	Music chalna

(b) Point	Started service	Bound service
Start kaise	startService()	bindService()
Kab tak chalti hai	stopSelf() / stopService()	Saare clients unbind
Communication	Caller se nahi	IBinder se client-server
Key callback	onStartCommand()	onBind()

(c) Point	Explicit intent	Implicit intent
Target	Class ka naam	Action batate hain
Use	Apne app ke andar	Doosre apps use karna
Kaun resolve karta hai	Tum	System, intent filters se

Conclusion: Sahi component aur intent type chunne se app simple aur secure rehta hai.

Quick Revision Sheet (exam se pehle raat ko padho)

Term	Ye yaad rakho
4 components	Activity, Service, Broadcast Receiver, Content Provider
Context	App environment ka handle. getApplicationContext() vs this
Activity life cycle	onCreate, onStart, onResume | onPause, onStop, onRestart, onDestroy
Home / Back / Rotate	Home: onPause, onStop. Back: + onDestroy. Rotate: destroy phir recreate
Save state	onSaveInstanceState(Bundle) aur onCreate() me restore
Launch modes	standard, singleTop, singleTask, singleInstance
Service types	Foreground (notification), Background, Bound
Service callbacks	onCreate, onStartCommand, onBind, onUnbind, onDestroy
Start / stop service	startService() / stopService() ya stopSelf()
Service rules	Android 8+ background limits. IntentService deprecated. Explicit intents use karo
Intent types	Explicit (class named) aur Implicit (action + data)
Intent parts	Component, Action, Data, Category, Extras, Flags
Pass data	putExtra() aur getIntent().getStringExtra()
Broadcast	sendBroadcast(), sendOrderedBroadcast(), onReceive()
Receiver registration	Manifest (static) ya registerReceiver() (dynamic); unregister karo
Manifest settings	uses-permission, uses-feature, application, activity, service, receiver, provider
android:exported	Android 12+ target par intent-filter ke saath zaroori
Intent filter	action + category + data; teeno tests pass hone chahiye
Launcher filter	MAIN + LAUNCHER
Permission types	Normal, Dangerous (runtime), Signature, Special
Runtime permission	Android 6.0+: checkSelfPermission, requestPermissions, onRequestPermissionsResult

Yaad rakhne ke tricks

Activity life cycle: C-S-R-P-S-R-D = Create, Start, Resume, Pause, Stop, Restart, Destroy.
Visible = Start se Stop. Foreground = Resume se Pause. Entire = Create se Destroy.
Intent parts: C-A-D-C-E-F = Component, Action, Data, Category, Extras, Flags.
Permission types: N-D-S-S = Normal, Dangerous, Signature, Special.
Pause = aanshik visible. Stop = hidden.

Common Mistakes jo Students Karte Hain

- **onPause()** (aanshik visible) aur **onStop()** (visible nahi) ko mix karna.
- Activity ya service ko **manifest me declare karna bhool jana**. App crash hota hai.
- **onReceive()** ya service ke main thread par heavy kaam karna, jisse ANR aata hai.
- Dangerous permissions ke liye sirf manifest me declare karna aur **run time par request na karna**.
- Life-cycle diagram na banana. Isme sabse zyada marks hote hain.

Self-Test (bina dekhe likho, har question 5 marks)

- Activity life cycle draw karke explain karo. Home button dabane par kya call hota hai?
- Service kya hai? Started aur bound services explain karo.
- Doosri activity start karne aur usse naam pass karne ka program likho.
- Intent Filter kya hai? Launcher activity ka filter likho.
- Android permissions ke types explain karo. Dangerous permission kaise maangi jaati hai?

Self-Test Answers (likhne ke baad check karo)

1. Figure 3 banao. Methods: onCreate, onStart, onResume, onPause, onStop, onRestart, onDestroy. Home button: **onPause(), phir onStop()**. Wapas aane par: onRestart, onStart, onResume.

2. Service bina UI ke long-running background kaam karti hai. Started: startService(), stopSelf()/stopService() tak chalti hai. Bound: bindService(), IBinder se client-server, saare clients unbind hone par band. Figure 5 banao.

3. Intent intent = new Intent(MainActivity.this, SecondActivity.class); intent.putExtra("name", "Amrit"); startActivity(intent); SecondActivity me: getIntent().getStringExtra("name"). SecondActivity ko manifest me declare karo.

4. Intent filter batata hai ki component kaunse intents receive kar sakta hai. Launcher: <intent-filter> <action android:name="android.intent.action.MAIN" /> <category android:name="android.intent.category.LAUNCHER" /> </intent-filter>.

5. Normal (install par auto), Dangerous (runtime), Signature (same certificate), Special (Settings screen). Dangerous: manifest me declare, checkSelfPermission() se check, requestPermissions() se request, onRequestPermissionsResult() handle.

> **Unit II complete!**
> Agar 5 me se 4 ya zyada sahi hue to **Unit III** par jao. Nahi to Part A me jo topics miss hue unhe dobara padho.